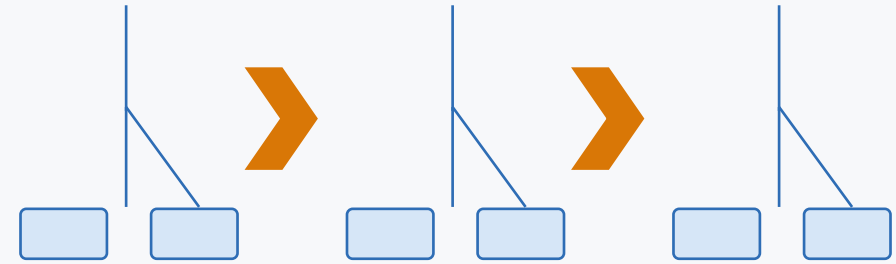


Boosting Methods and XGBoost

From weak learners to a regularized, scalable boosting system



Sequential error correction

Outline

- Why boosting? Contrast with bagging and random forests
- AdaBoost intuition: reweighting hard cases
- Gradient boosting: fit new trees to the current loss / residual pattern
- XGBoost: regularization, shrinkage, subsampling, and efficient tree search
- Key tuning parameters and practical workflow in R
- Banknote classification example, exercise, and solution structure

Bagging

many trees in parallel → average / vote

Boosting

small trees added sequentially to fix current errors



Why boosting?

Core idea

Instead of averaging many independent high-variance trees, boosting builds a sequence of small trees where each new tree focuses on what the current ensemble still gets wrong.

Think: “add another tree that repairs the remaining error.”

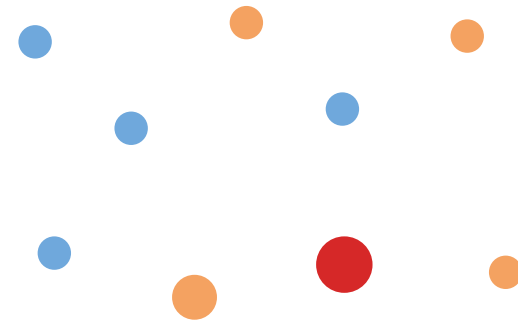
Bagging vs. Boosting		
Training style	parallel	sequential
Primary goal	reduce variance	reduce bias + variance
Learners	often deep trees	often shallow trees
Sensitivity to noise	lower	can be higher
Typical examples	bagging, RF	AdaBoost, GBM, XGBoost

AdaBoost intuition

Reweight the hard cases

- Start with equal weights on all training observations.
- Fit a weak learner (often a decision stump).
- Increase the weights of misclassified observations; decrease the weights of correctly classified observations.
- Fit the next learner to this new weighted problem.
- Take a weighted vote of all learners.

Observation weights



larger points = currently harder cases

Limitation: can be sensitive to label noise and outliers.

Gradient boosting

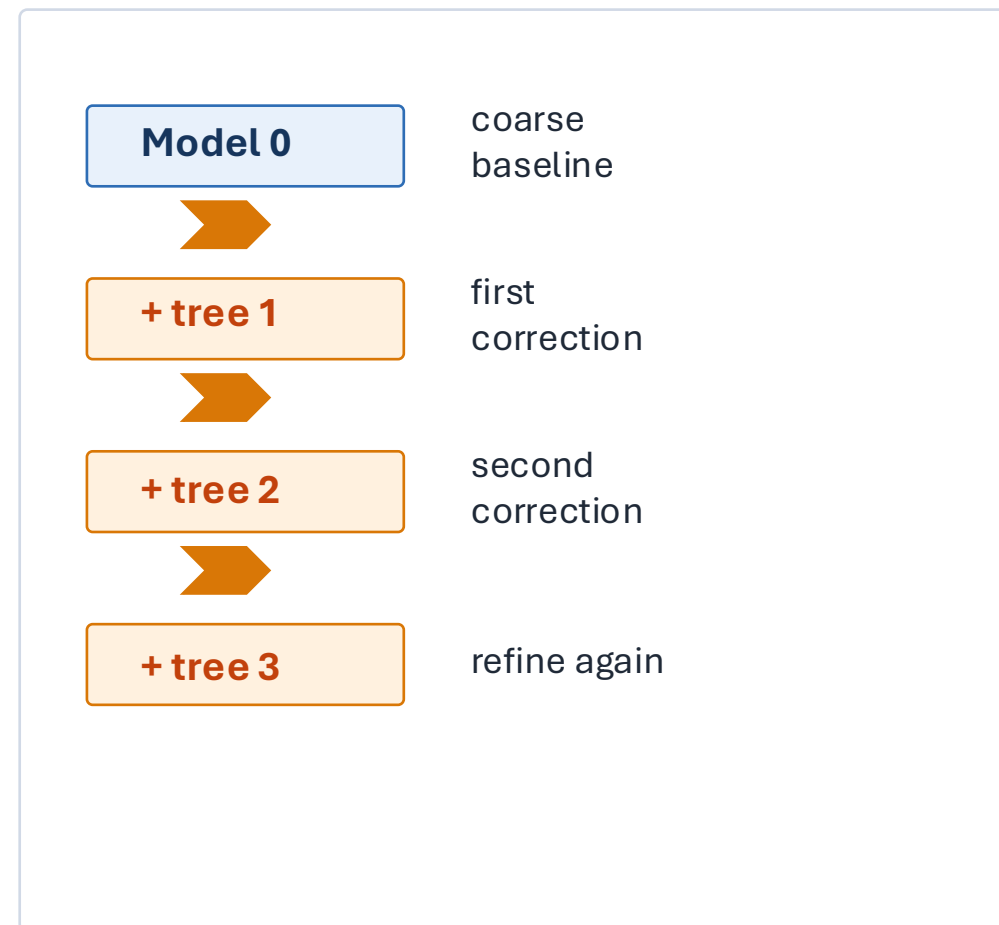
Fit a new tree to the current loss gradient

For squared-error regression, the negative gradient is the residual, so the next tree is literally fitted to what remains unexplained.

$$F_m(x) = F_{m-1}(x) + u h_m(x)$$

$$h_m(x) \approx - \left[\frac{\partial L(y, F(x))}{\partial F(x)} \right]_{F=F_{m-1}}$$

u = learning rate / shrinkage



What makes XGBoost different?

$$\text{Obj} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|_2^2$$

Objective = Loss + Regularization

- Regularized objective controls tree complexity.
- Shrinkage (eta) slows each update and often improves generalization.
- Row subsampling and column subsampling add randomness, similar in spirit to bagging and random forests.
- Efficient split finding plus sparsity-aware handling makes training fast on large tabular data.
- Built-in importance measures and strong performance on structured data.

Regularization γ and λ penalize overly complex trees

Subsampling subsample and colsample_bytree
reduce overfitting

Efficiency parallelized split search, cache-aware design

Missing values learns a default direction for a missing split path

Key XGBoost tuning parameters

Parameter	Role	If increased too much
eta	step size / learning rate	underfitting if too tiny; unstable if too large
nrounds	number of boosting iterations	overfitting and longer runtime
max_depth	tree complexity	fits noise and interactions too aggressively
subsample	row sampling per tree	too low can miss signal
colsample_bytree	feature sampling per tree	too low can weaken learners
min_child_weight	minimum weight in a child node	too small can overfit
gamma	minimum loss reduction for a split	too large can block useful splits

Practical workflow in R

Graduate-level workflow for tabular classification

```
1 library(caret)
2 library(xgboost)
3
4 set.seed(123)
5 idx <- createDataPartition(y, p = 0.75, list = FALSE)
6 train.data <- data[idx, ]
7 test.data <- data[-idx, ]
8
9 grid <- expand.grid(
10   nrounds = c(50, 100),
11   max_depth = c(2, 4, 6),
12   eta = c(0.05, 0.1, 0.3),
13   gamma = 0,
14   colsample_bytree = c(0.8, 1.0),
15   min_child_weight = 1,
16   subsample = c(0.8, 1.0)
17 )
18
19 fit <- train(class ~ ., data = train.data,
20             method = "xgbTree",
21             trControl = trainControl(method = "cv", number =
22             5),
23             tuneGrid = grid)
```

- 1. Define the positive class and split the data.
- 2. Tune with CV rather than trusting defaults.
- 3. Inspect best hyperparameters and held-out metrics.
- 4. Plot feature importance, then interpret cautiously.
- 5. Compare against a simpler baseline (e.g., GBM or RF).

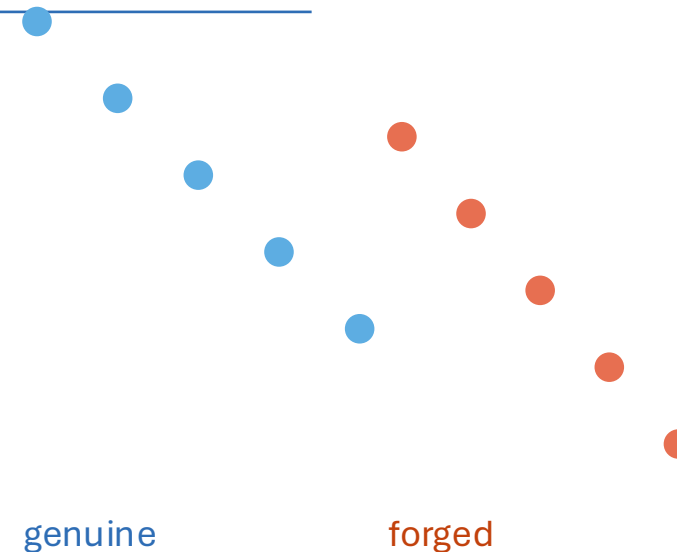
Example: Banknote authentication

Dataset used in the exercise

- 1,372 observations; 4 continuous predictors extracted from wavelet-transformed images.
- Binary response: class = 1 (forged) vs 0 (genuine).
- Why it works well for teaching: low-dimensional, numeric, and strong signal with limited preprocessing.

Predictors	
variance	skewness
curtosis	entropy

Class separation (conceptual)



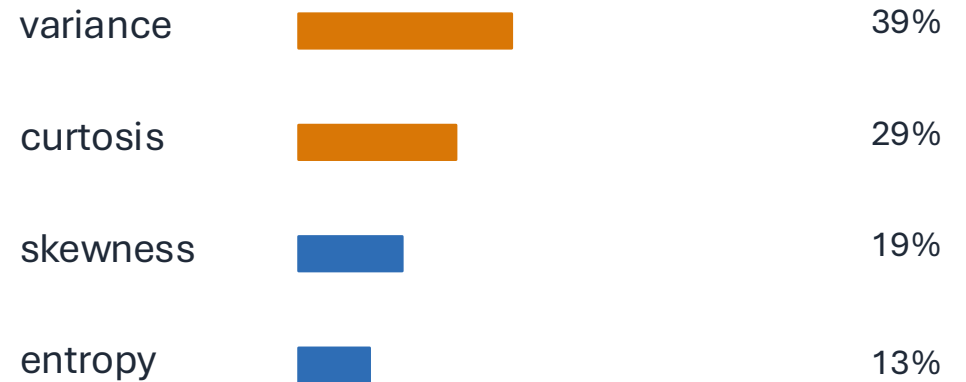
The exercise asks students to compare GBM with XGBoost on the same 75/25 split.

Model evaluation and interpretation

For classification, report:

- Confusion matrix
- Sensitivity (positive class = forged)
- Specificity
- Overall accuracy
- Cross-validated best tuning parameters

Feature importance



Interpret importance as model-specific and relative, not causal.

Advantages and limitations

Why practitioners like XGBoost

- Strong performance on structured / tabular data
- Flexible loss functions for regression and classification
- Regularization and subsampling help control overfitting
- Works well with moderate feature engineering
- Often competitive with much more complicated models

What to watch out for

- More tuning effort than a plain tree or linear model
- Sequential boosting can overfit noisy labels if unchecked
- Interpretability is weaker than a single tree
- Data leakage in time series or grouped data can make results look unrealistically good
- Feature importance is useful but not a substitute for domain reasoning